CITY CARE

Documentación completa del apartado 2 1 Autenticación

Backend AdonisJS 7 con MySQL OpenAPI y Orval

Versión documental 2.0
31 de octubre de 2026

Estado del apartado 2.1: COMPLETO Y VERIFICADO

Este documento explica qué se implementó, cómo funciona, cómo se valida y cómo operar el módulo de autenticación.

Propósito y alcance

El apartado 2.1 cubre la autenticación y la seguridad de las sesiones de City Care. Su objetivo es permitir el acceso mediante correo y contraseña, proteger las rutas privadas, cerrar sesiones, limitar intentos repetidos, impedir el acceso de cuentas inactivas, caducar los tokens y devolver errores uniformes para el frontend.

La implementación quedó cerrada con nueve pruebas funcionales aprobadas. También se verificaron TypeScript, ESLint, el build, las rutas registradas y las seis migraciones aplicadas en MySQL. SQLite se utiliza únicamente como base efímera en memoria durante las pruebas automatizadas; el entorno de desarrollo continúa trabajando con MySQL.

Resumen de cumplimiento

Punto del 2.1	Estado	Evidencia principal
Login con correo y contraseña	Completo	Valida credentials y emite un access token.
Hash de contraseñas	Completo	password_hash se genera y verifica con el servicio hash de AdonisJS.
Perfil y logout	Completo	Rutas protegidas para consultar perfil y eliminar el token actual.
Rate limiting	Completo	Cinco intentos por minuto y bloqueo de quince minutos por IP y correo.
Bloqueo de usuarios inactivos	Completo	El login responde 403 antes de emitir un token.
Invalidación de tokens al desactivar	Completo	El middleware revoca todas las sesiones al detectar is_active false.
Expiración de tokens	Completo	ACCESS_TOKEN_EXPIRES_IN con valor predeterminado de siete días.
Credencial inicial fuera del código	Completo	ADMIN_INITIAL_PASSWORD se obtiene del entorno y exige doce caracteres.
Errores estandarizados	Completo	Estructura error code message y details para 400 401 403 422 y 429.

Contenido

1. Arquitectura y flujo general
2. Modelo de datos y migraciones
3. Implementación de cada punto del apartado 2.1
4. Endpoints y contratos HTTP
5. OpenAPI Swagger y Orval
6. Pruebas y evidencia de verificación

7. Operación del módulo
8. Archivos principales y responsabilidades
9. Alcance cerrado y trabajo posterior

1 Arquitectura y flujo general

El módulo separa el perfil del usuario de sus credenciales. Los controladores coordinan las solicitudes HTTP, AuthService valida identidad y estado, los middleware protegen las rutas, Lucid administra la persistencia y los transformadores limitan los datos que se exponen al cliente.

1.1 Componentes

Capa	Responsabilidad
Rutas	Exponen login, profile, logout y alta administrativa; asignan middleware.
Validadores	Comprueban email, contraseña y cuerpos administrativos mediante VineJS.
Controladores	Coordinan autenticación, emisión o eliminación de tokens y serialización.
AuthService	Busca credentials, verifica password_hash, aplica rate limiting y controla is_active.
Middleware auth	Autentica el Bearer token, vuelve a validar is_active y revoca sesiones.
Middleware admin	Carga role y permite operaciones administrativas únicamente al slug admin.
Modelos Lucid	Representan users, credentials, roles, dependencies y auth_access_tokens.
Transformer	Devuelve únicamente los campos públicos del usuario y su rol.
Exception handler	Convierte errores en una respuesta JSON uniforme.

1.2 Secuencia del inicio de sesión

1. El cliente envía email y password a POST /api/v1/auth/login.
2. loginValidator valida formato, longitud del correo y presencia de la contraseña.
3. AccessTokensController entrega email, password e IP a AuthService.login.
4. El limitador crea una clave con IP y correo y controla los intentos fallidos.
5. AuthService busca la credencial en credentials y verifica password_hash.
6. El servicio recupera users mediante credentials.user_id y comprueba is_active.
7. El controlador carga role, crea un access token con expiración y serializa la respuesta.
8. El cliente conserva el token y lo envía como Authorization Bearer en rutas protegidas.

```
Cliente -> loginValidator -> AccessTokensController -> AuthService
        -> credentials -> users -> role -> access token -> respuesta data
```

1.3 Autenticación y autorización

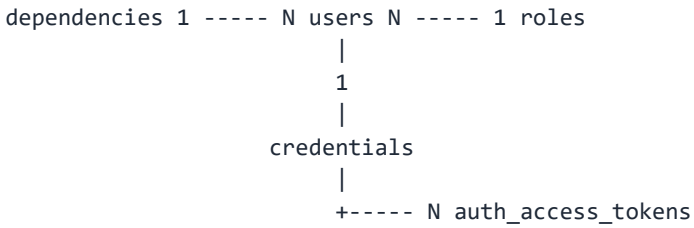
La autenticación determina quién es el usuario mediante un access token. La autorización determina qué puede hacer: AdminMiddleware carga la relación role y exige roles.slug igual a admin. El uso del slug evita depender de identificadores numéricos que pueden variar entre bases de datos.

2 Modelo de datos y migraciones

La autenticación utiliza cinco tablas funcionales. Las seis migraciones del proyecto se encuentran en estado completed en MySQL; una de ellas agrega la relación de roles sobre users.

Tabla	Campos relevantes	Propósito
users	id dependency_id role_id email is_active	Perfil, dependencia, rol y estado de la cuenta.
credentials	user_id email password_hash	Datos privados usados para iniciar sesión.
roles	id name slug	Nivel de autorización del usuario.
dependencies	id name description	Área organizacional a la que pertenece el usuario.
auth_access_tokens	tokenable_id hash abilities expires_at	Sesiones emitidas para cada usuario.

2.1 Relaciones



- Cada usuario pertenece a una dependencia y a un rol.
- Cada usuario tiene una credencial identificada por user_id único.
- Un usuario puede tener varios tokens para sesiones independientes.
- Al eliminar un usuario, sus credenciales y tokens se eliminan mediante cascada.

2.2 Migraciones verificadas

Orden	Migración	Estado
1	create_dependencies_table	completed lote 1
2	create_users_table	completed lote 1
3	create_access_tokens_table	completed lote 1
4	create_credentials_table	completed lote 1

Orden	Migración	Estado
5	create_roles_table	completed lote 2
6	alter_users_table	completed lote 3

MySQL es la base de desarrollo y conserva la información real. Durante npm test, AdonisJS cambia a SQLite en memoria, ejecuta las mismas migraciones, corre las pruebas y revierte las tablas al terminar. Este aislamiento impide que las pruebas modifiquen city-care en MySQL.

3 Implementación de los puntos del apartado 2 1

3.1 Login con correo y contraseña

POST /api/v1/auth/login recibe email y password. El correo se normaliza con trim, debe tener formato válido y un máximo de 150 caracteres. AuthService consulta credentials por email y utiliza el servicio hash para verificar la contraseña. Si la combinación es válida, recupera users, carga role y crea el token.

Para no revelar si un correo existe, una credencial inexistente y una contraseña incorrecta producen exactamente el mismo error E_INVALID_CREDENTIALS con estado 400.

3.2 Hash de contraseñas

Las contraseñas nunca se guardan en users ni en texto plano. Al crear una cuenta o preparar el administrador inicial, hash.make genera password_hash. Durante el login, hash.verify compara la contraseña recibida con ese hash. La base no permite recuperar la contraseña original; si se pierde, debe restablecerse.

3.3 Perfil y logout

GET /api/v1/account/profile requiere un Bearer token válido, carga el rol y devuelve UserTransformer. POST /api/v1/account/logout elimina solamente el token usado en esa solicitud. Otros tokens del mismo usuario permanecen activos, salvo que la cuenta sea desactivada o se ejecute una revocación global.

3.4 Rate limiting

AuthService usa el limitador con cinco solicitudes durante un minuto y blockDuration de quince minutos. La clave login_IP_email separa los intentos por dirección y cuenta. El sexto intento repetido devuelve 429 E_TOO_MANY_REQUESTS. El almacén actual es memory, adecuado para una sola instancia de desarrollo; un despliegue con varias instancias requerirá un almacén compartido.

3.5 Bloqueo de usuarios inactivos

Después de validar el hash y antes de emitir el token, AuthService comprueba users.is_active. Cuando el valor es false, elimina todos los tokens del usuario y responde 403 E_ACCOUNT_INACTIVE. Por lo tanto, una cuenta desactivada no recibe una nueva sesión.

3.6 Invalidación de tokens al desactivar

El control también se ejecuta en AuthMiddleware. Si una cuenta tenía un token previo y luego se cambia a inactiva, el siguiente intento de usar una ruta protegida provoca la revocación de todas sus sesiones y devuelve 403. AuthService.revokeAllTokens centraliza esta operación.

3.7 Expiración configurable de tokens

DbAccessTokensProvider configura expiresIn con ACCESS_TOKEN_EXPIRES_IN. Si la variable no existe, se utiliza 7d. La prueba funcional confirma que el campo expires_at se genera aproximadamente siete días después de la emisión.

```
ACCESS_TOKEN_EXPIRES_IN=7d
```

3.8 Credencial inicial fuera del código

AdminSeeder ya no contiene una contraseña fija. Para crear por primera vez admin@citycare.com exige ADMIN_INITIAL_PASSWORD con al menos doce caracteres. Si el administrador y su credencial ya existen, ejecutar nuevamente el seeder no reemplaza la contraseña. El valor real debe permanecer solamente en el archivo .env local o en el gestor de secretos del entorno.

```
ADMIN_INITIAL_PASSWORD=<secreto de al menos 12 caracteres>
node ace db:seed
```

3.9 Errores estandarizados

HttpExceptionHandler transforma los errores conocidos en un objeto uniforme. Los errores de validación incluyen details; los errores internos ocultan información sensible.

```
{
  "error": {
    "code": "E_INVALID_CREDENTIALS",
    "message": "Credenciales inválidas"
  }
}
```

HTTP	Código	Situación
400	E_INVALID_CREDENTIALS	Correo inexistente o contraseña incorrecta.
401	E_UNAUTHORIZED_ACCESS	Token ausente, inválido, vencido o eliminado.
403	E_ACCOUNT_INACTIVE	Cuenta desactivada.
403	Autorización administrativa	Usuario autenticado sin rol admin.
422	E_VALIDATION_ERROR	Cuerpo incompleto o con formato inválido.
429	E_TOO_MANY_REQUESTS	Límite de intentos superado.

4 Endpoints y contratos HTTP

Método	Ruta	Acceso	Resultado
POST	/api/v1/auth/login	Público	Usuario público y token.
GET	/api/v1/account/profile	Bearer	Perfil del usuario y rol.

Método	Ruta	Acceso	Resultado
POST	/api/v1/account/logout	Bearer	Elimina el token actual.
POST	/api/v1/admin/users	Bearer y admin	Crea perfil y credencial en una transacción.

4.1 Iniciar sesión

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "admin@citycare.com",
  "password": "<contraseña>"
}
```

```
{
  "data": {
    "user": {
      "id": 1,
      "email": "admin@citycare.com",
      "isActive": true,
      "role": { "id": 1, "name": "Administrador", "slug": "admin" }
    },
    "token": "<access_token>"
  }
}
```

4.2 Consultar perfil

```
GET /api/v1/account/profile
Authorization: Bearer <access_token>
```

La respuesta se encuentra dentro de data e incluye id, dependencyId, roleId, nombres, correo, datos opcionales, isActive, fechas, initials y role. No incluye password_hash ni el hash del token.

4.3 Cerrar sesión

```
POST /api/v1/account/logout
Authorization: Bearer <access_token>

{ "message": "Logged out successfully" }
```

4.4 Crear una cuenta administrativa

Aunque pertenece al apartado de administración de usuarios, esta ruta participa en el ciclo de autenticación porque crea credentials. Requiere middleware auth y admin. NewAccountController genera el hash antes de abrir la transacción y después crea users y credentials con el mismo cliente transaccional.

```
POST /api/v1/admin/users
Authorization: Bearer <token_admin>
Content-Type: application/json

{
  "dependencyId": 1,
  "roleId": 2,
  "firstName": "Ana",
  "lastName": "López",
  "email": "ana@ejemplo.com",
  "isActive": true,
  "password": "Password123!",
  "passwordConfirmation": "Password123!"
}
```

4.5 Uso seguro del token

- El valor completo del token se entrega únicamente en la respuesta de login.
- La base almacena el hash del token, no el valor reutilizable por el cliente.
- El frontend debe enviarlo en Authorization: Bearer y retirarlo al cerrar sesión.
- No se debe incluir el token en URLs, registros, capturas ni documentación pública.

5 OpenAPI Swagger y Orval

AutoSwagger genera el contrato desde rutas, anotaciones e interfaces. GET /swagger devuelve YAML normalizado y GET /docs presenta Swagger UI. La normalización elimina required vacíos, convierte propiedades obligatorias al formato OpenAPI, marca requestBody como requerido y agrega un ejemplo completo de login con email y password.

Recurso	Ubicación o comando	Uso
Contrato vivo	GET /swagger	Especificación OpenAPI normalizada.
Interfaz	GET /docs	Exploración y prueba manual de endpoints.
Contrato versionado	openapi/city-care.v1.yaml	Entrada estable para Orval.
Configuración	orval.config.ts	Genera cliente fetch en modo single.
SDK generado	sdk/generated/city-care.ts	Tipos y funciones para el frontend.
Regeneración	npm run openapi:generate	Actualiza el SDK desde el contrato.

5.1 Corrección del campo password en Swagger

El esquema LoginRequest siempre contenía email y password, pero el ejemplo automático de AutoSwagger omitía password por tratarlo como campo sensible. Swagger UI priorizaba ese ejemplo y parecía mostrar un cuerpo incompleto. Se definió un ejemplo explícito sin credenciales reales; ahora la interfaz presenta ambos campos y Orval conserva password como obligatorio.

```
{
  "email": "usuario@citycare.com",
  "password": "TuPassword123!"
}
```

Los valores anteriores son únicamente ilustrativos y no deben interpretarse como una cuenta válida.

5.2 Flujo de sincronización

1. Iniciar el backend y verificar http://localhost:3333/swagger.
2. Actualizar openapi/city-care.v1.yaml desde el contrato vivo.
3. Ejecutar npm run openapi:generate.
4. Comprobar que LoginRequest contiene email y password.
5. Ejecutar npm run typecheck y npm run lint.
6. Copiar o consumir el SDK generado desde el frontend definitivo.

6 Pruebas y evidencia de verificación

El grupo Autenticación 2.1 contiene nueve pruebas funcionales. Cada ejecución levanta SQLite en memoria, aplica las seis migraciones y elimina la información al terminar. La evidencia del 31 de octubre de 2026 registró nueve pruebas aprobadas en aproximadamente dos segundos.

Prueba	Resultado esperado	Estado
Login válido	HTTP 200, token y expires_at cercano a siete días.	Aprobada
Perfil	HTTP 200 con id y email correctos.	Aprobada
Logout	El token deja de acceder y recibe 401.	Aprobada
Login inactivo	HTTP 403 y ninguna sesión emitida.	Aprobada
Token de cuenta desactivada	HTTP 403 y revocación de todos los tokens.	Aprobada
Credenciales inválidas	HTTP 400 con respuesta uniforme.	Aprobada
Rate limiting	Cinco errores y sexto intento con HTTP 429.	Aprobada
Validación	HTTP 422 con error y details.	Aprobada
Token ausente	HTTP 401 E_UNAUTHORIZED_ACCESS.	Aprobada

6.1 Comprobaciones técnicas

Comando	Resultado verificado
npm test	9 pruebas aprobadas

Comando	Resultado verificado
npm run typecheck	Sin errores TypeScript
npm run lint	Sin errores ESLint
npm run build	Compilación completada
node ace list:routes	Siete rutas totales y cuatro endpoints API
node ace migration:status	Seis migraciones completed en MySQL
npm run openapi:generate	Cliente Orval generado correctamente

```
npm test
npm run typecheck
npm run lint
npm run build
node ace list:routes
node ace migration:status
npm run openapi:generate
```

6.2 Prueba manual en Swagger

1. Ejecutar npm run dev y abrir http://localhost:3333/docs.
2. Abrir POST /api/v1/auth/login y seleccionar Try it out.
3. Introducir una cuenta válida y ejecutar la solicitud.
4. Copiar el token devuelto sin publicarlo ni guardarlo en documentación.
5. Usar Authorize o el encabezado Bearer para consultar profile.
6. Ejecutar logout y comprobar que el mismo token ya responde 401.

7 Operación del módulo

7.1 Preparar una instalación

1. Copiar .env.example como .env y configurar MySQL.
2. Generar APP_KEY según el flujo de AdonisJS.
3. Definir ACCESS_TOKEN_EXPIRES_IN; el valor recomendado inicial es 7d.
4. Definir ADMIN_INITIAL_PASSWORD solamente si todavía se creará el administrador inicial.
5. Ejecutar las migraciones y después el seeder.
6. Iniciar el servidor y verificar Swagger.

```
npm ci
node ace migration:run
node ace db:seed
npm run dev
```

7.2 Cuenta administrativa

El seeder prepara el correo admin@citycare.com, los roles admin y user y la dependencia Administración. La contraseña concreta no debe escribirse en este documento. En el entorno local actual fue restablecida directamente para permitir la prueba, pero debe tratarse como secreto y cambiarse si el archivo se comparte o el entorno se publica.

7.3 Cambio o pérdida de contraseña

password_hash no puede descifrarse. Mientras no exista un endpoint de recuperación, el restablecimiento debe realizarse mediante una operación administrativa controlada que vuelva a generar el hash. No se debe escribir texto plano directamente en credentials.password_hash.

7.4 Desactivación de cuentas

Cuando el CRUD administrativo implemente el cambio de estado, debe establecer users.is_active en false y llamar a AuthService.revokeAllTokens dentro del mismo caso de uso. El middleware actual actúa como segunda barrera y revoca sesiones cuando se intenta reutilizar un token.

7.5 Consideraciones para producción

- Sustituir el limitador memory por un almacén compartido si hay varias instancias.
- Guardar APP_KEY, ADMIN_INITIAL_PASSWORD y credenciales de MySQL en un gestor de secretos.
- Configurar CORS únicamente para los dominios permitidos y servir la API mediante HTTPS.
- Definir monitoreo, respaldos y auditoría de cambios administrativos.
- Implementar recuperación de contraseña y cierre global de sesiones como módulo posterior.

8 Archivos principales y responsabilidades

Archivo	Responsabilidad
start/routes.ts	Rutas de API, Swagger y normalización del contrato.
app/services/auth_service.ts	Credenciales, limitador, inactivos y revocación global.
app/controllers/access_tokens_controller.ts	Login, emisión de token y logout.
app/controllers/profile_controller.ts	Perfil autenticado.
app/middleware/auth_middleware.ts	Bearer token y segunda validación de is_active.
app/middleware/admin_middleware.ts	Autorización por role.slug admin.
app/exceptions/handler.ts	Formato uniforme de errores.
app/validators/user.ts	Validación de login y alta de usuarios.
app/models/user.ts	Relaciones y proveedor de tokens con expiración.
app/models/credential.ts	Relación privada de credenciales con user.
app/transformers/user_transformer.ts	Representación pública del usuario.

Archivo	Responsabilidad
database/seeders/admin_seeder.ts	Roles, dependencia y administrador inicial seguros.
config/database.ts	MySQL en desarrollo y SQLite en memoria para pruebas.
config/auth.ts	Guard api basado en access tokens.
config/limiter.ts	Almacén del limitador.
tests/functional/auth.spec.ts	Nueve pruebas del cierre 2.1.
openapi/city-care.v1.yaml	Contrato OpenAPI versionado.
orval.config.ts	Configuración del generador de cliente.
sdk/generated/city-care.ts	Tipos y funciones fetch generadas.

9 Alcance cerrado y trabajo posterior

El apartado 2.1 se considera cerrado porque todos sus criterios están implementados y probados. Esto no significa que la versión 1.0 completa de City Care esté terminada. Los siguientes trabajos pertenecen a otros apartados y no bloquean el cierre técnico del login.

Trabajo posterior	Apartado	Motivo
CRUD completo de usuarios	2.2	Faltan listado, detalle, edición y baja lógica.
Permisos granulares	2.3	Actualmente la autorización administrativa usa slug admin.
CRUD de dependencias	2.4	Se requiere para formularios y administración organizacional.
Bearer centralizado en frontend	2.5 y 2.6	El SDK debe adjuntar el token y manejar la URL por entorno.
Pruebas administrativas	2.7	Faltan pruebas específicas del CRUD y su transacción.
Recuperación de contraseña	Etapa posterior	Requiere tokens temporales, auditoría y política propia.

Conclusión

City Care cuenta con un módulo de autenticación funcional y verificable: valida credenciales separadas del perfil, protege rutas mediante tokens, controla cuentas inactivas, revoca sesiones, limita intentos, aplica expiración configurable, evita contraseñas fijas en el código y entrega errores compatibles con el frontend. El contrato OpenAPI y el SDK Orval reflejan el cuerpo completo de login. Con esta base, el desarrollo puede continuar hacia el CRUD administrativo y la integración integral del frontend sin reabrir el apartado 2.1.